

Chapitre 4 - Requêtes en SQL – SELECT

I.	Introduction	1
II.	Deux types de requêtes SQL.....	1
III.	Requête SELECT.....	2
1.	La projection	2
2.	La sélection - bases	3
3.	La sélection - compléments	4
4.	La jointure.....	6
5.	Trier les données affichées.....	7
6.	Fonctions de regroupement.....	8
7.	Regroupement.....	10
8.	Regroupement avec condition.....	12
9.	Conditions d'appartenance / non appartenance à un ensemble	13
10.	Clause DISTINCT.....	15

I. Introduction

Il existe 2 moyens pour réaliser des requêtes dans un SGBD :

- QBE (Query By Example),
- SQL (Structured Query Language)

QBE est un mode assisté, convivial, qui s'adresse aux utilisateurs néophytes. Il est différent d'un SGBD à l'autre (voire absent !).

SQL est un langage normalisé (standardisé), qui impose un apprentissage mais offre l'avantage d'être universel.

II. Deux types de requêtes SQL

On distingue les requêtes utilisant le LDD (Langage de Description de Données) de celles utilisant le LID (Langage d'Interrogation des Données) et le LMD (Langage de Manipulation de Données)

Le LDD sert notamment à créer les tables, les modifier, les supprimer, et à placer ou enlever les contraintes d'intégrité.

Les LID et LMD contiennent ordres SQL :

- SELECT pour interroger (LID),
- INSERT pour ajouter un enregistrement dans une table (LMD),
- DELETE pour supprimer un ou plusieurs enregistrements dans une table (LMD),
- UPDATE pour modifier un ou plusieurs enregistrements dans une table (LMD),

III. Requête SELECT

1. La projection

Syntaxe :

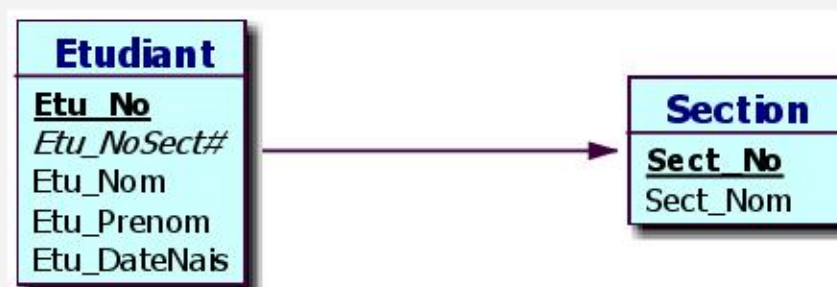
```
SELECT ColonnesAVoir  
FROM TablesUtilisées
```

ColonnesAVoir contient les colonnes que l'on veut voir figurer dans le résultat de la requête.
TablesUtilisées indique d'où viennent les données (colonnes) utilisées.

Si l'on veut afficher plusieurs colonnes (*ColonnesAVoir*), il faut les séparer par une virgule. Il en est de même pour les tables (*TablesUtilisées*).

Pour voir toutes les colonnes d'une table, utiliser * à la place des noms de colonnes

Exemple : Afficher les noms et prénoms des étudiants :



```
SELECT Etu_Nom, Etu_Prénom  
FROM Etudiant
```

La *projection* consiste à afficher certaines colonnes (ici deux de la table Etudiant). Etu_Nom et Etu_Prénom sont donc les champs projetés

Le résultat aura l'allure suivante :

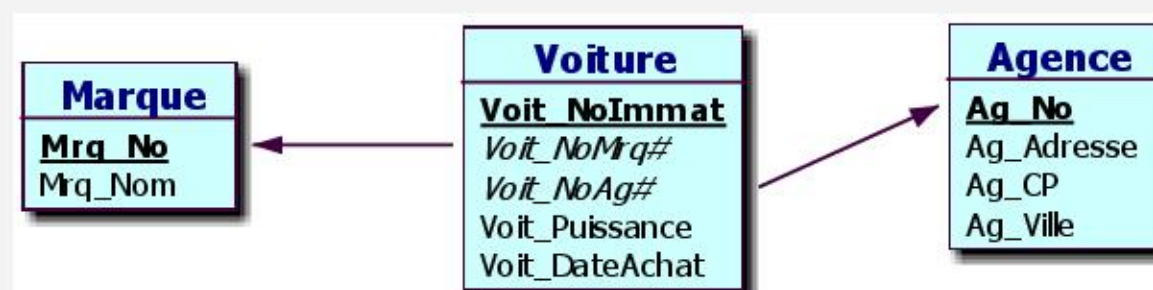
Etu_Nom	Etu_Prénom
DURAND	Robert
JOYAT	Jeannine
TRIOS	Caroline

...

Le résultat aura autant de lignes qu'il y a d'étudiants dans la table Etudiant.

Exercice

Parc véhicules d'agences immobilières



Produire les requêtes suivantes :

R1 : Liste des marques de voitures

R2 : Liste des véhicules (n° d'immatriculation et puissance)

R3 : Liste des agences de Grenoble

R4 : Que produit la requête suivante ?
SELECT Voit_Puissance, Voit_DateAchat
FROM Voiture

Correction

R1 : SELECT Mrq_No, Mrq_Nom
FROM Marque
R2 : SELECT Voit_NoImmat, Voit_Puissance
FROM Voiture
R3 : Impossible pour l'instant. Il faut un critère !
R4 : Liste des puissances et dates d'achat de toutes les voitures

2. La sélection - bases

Syntaxe :
SELECT *ColonnesAVoir*
FROM *TablesUtilisées*
WHERE *Conditions*

Condition contient des critères que les lignes affichées respecteront.

Si l'on veut utiliser plusieurs conditions (critères), il faudra les relier par des « *et* » ou bien des « *ou* », donc des AND ou bien des OR.

Exemple :

R1 : Liste des étudiants prénommés Robert
R2 : Afficher les noms des étudiants prénommés Robert ou Jeanine :



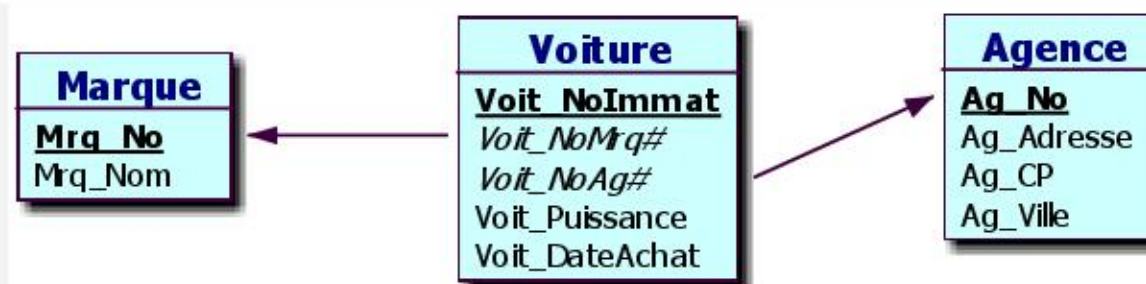
R1 : SELECT Etu_No, Etu_Nom
FROM Etudiant
WHERE Etu_Prenom = "Robert"

R2 : SELECT Etu_Nom
FROM Etudiant
WHERE Etu_Prenom = "Robert"
OR Etu_Prenom = "Jeannine"

La *sélection* consiste à n'afficher qu'une partie des lignes de la table Etudiant.

Exercice

Parc véhicules d'agences immobilières



Produire les requêtes suivantes :

R1 : Liste des voitures de 5 à 7 cv

R2 : Liste des véhicules achetés en 2000 (n° d'immatriculation et puissance)

R3 : Liste des agences de Grenoble et de St Marcellin

R4 : Que produit la requête suivante ?

```

SELECT Ag_No, Ag_Adresse
FROM Agence
WHERE Ag_Ville="Voiron"

```

Correction

R1 : SELECT Voit_NoImmat, Voit_Puissance
FROM Voiture
WHERE Voit_Puissance>=5 AND Voit_Puissance<=7

R2 : SELECT Voit_NoImmat, Voit_Puissance
FROM Voiture
WHERE Voit_DateAchat>=#1/1/2000#
AND Voit_DateAchat <=#31/12/2000#

R3 : SELECT Ag_No, Ag_Adresse
FROM Agence
WHERE Ag_Ville="Grenoble"
OR Ag_Ville="St Marcellin"

R4 : Liste des n° et adresse des agences de Voiron

3. La sélection - compléments

Critère du type « colonne renseignée » ou « colonne non renseignée »

Certains champs peuvent être facultatifs. On peut donc souhaiter placer un critère pour demander à ce qu'ils soient remplis ou non de la façon suivante :

Champ rempli : *Colonne* IS NOT NULL

Champ vide : *Colonne* IS NULL



Exemple :

Si le champs Etu_NoSect# dans Etudiant est facultatif, on peut obtenir :

R1 : La liste des étudiants sans affectation,

R2 : La liste des étudiants ayant une affectation, ainsi :


```
R1 : SELECT *  
FROM Etudiant  
WHERE Etu_NoSect# IS NULL
```

```
R2 : SELECT *  
FROM Etudiant  
WHERE Etu_NoSect# IS NOT NULL
```

Critère du type « colonne entre ceci et cela »

Pour établir un encadrement entre 2 valeurs (de type chaîne, numérique ou date), on a 2 solutions :

Faire appel à 2 comparaisons reliées par un ET : *Colonne* >= *ValeurMini* AND *Colonne* <= *ValeurMaxi*

Utiliser l'opérateur BETWEEN : *Colonne* BETWEEN *ValeurMini* AND *ValeurMaxi*

Exemple :

Liste des étudiants nés en 1997 :



```
SELECT Etu_Nom  
FROM Etudiant  
WHERE Etu_DateNais BETWEEN #1/1/1997# AND #31/12/1997#
```

ou bien

```
SELECT Etu_Nom  
FROM Etudiant  
WHERE Etu_DateNais >= #1/1/1997#  
AND Etu_DateNais <= #31/12/1997#
```

Critère du type « colonne ressemblant à ceci »

Quand on veut retenir des lignes en fonction d'un tel critère (type chaîne uniquement), on a 2 solutions :

Faire appel à 2 comparaisons reliées par un ET : *Colonne* >= *ValeurDébut* AND *Colonne* < *ValeurFin*

Utiliser l'opérateur LIKE : *Colonne* LIKE *ValeurDébut**

Exemple :

Liste des étudiants dont le nom commence par DUCH :



```
SELECT Etu_No, Etu_Nom
FROM Etudiant
WHERE Etu_Nom LIKE "DUCH*"
```

ou bien

```
SELECT Etu_No, Etu_Nom
FROM Etudiant
WHERE Etu_Nom >="DUCH" and Etu_Nom <"DUCI"
```

4. La jointure

Si la clause FROM contient 2 tables ou plus, toutes les lignes d'une table seront associées à toutes les lignes des autres tables : C'est le produit cartésien, jamais souhaité sauf très rare cas. Pour éviter cet effet, on ajoute des conditions dans la clause WHERE pour établir des liens entre chacune des tables : C'est la jointure entre 2 tables.

La jointure concerne toujours la clé primaire d'une table et la clé étrangère correspondante d'une autre table. Astuce à utiliser : La contrainte d'intégrité rapproche ces mêmes éléments.

La jointure est toujours exprimée ainsi :

CléPrimaire = CléEtrangère

Ou bien *CléEtrangère = CléPrimaire*

Exemple :

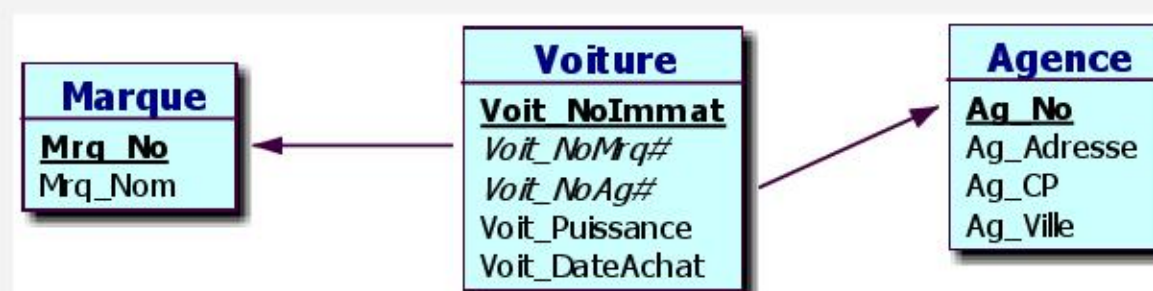
Liste des étudiants et de leurs sections :



```
SELECT Etu_No, Etu_Nom , Sect_Nom
FROM Etudiant, Section
WHERE Etu_NoSect# = Sect_No
```

Exercice

Parc véhicules d'agences immobilières



Produire les requêtes suivantes :

R1 : Liste des véhicules de marque *Renault*

R2 : Liste des véhicules de plus de 6 chevaux de l'agence n° 4 (n° d'immatriculation et puissance)

R3 : Liste des agences possédant au moins un véhicule de marque *Peugeot* (No et ville d'agence)

R4 : Liste des agences ne possédant pas de véhicule de marque *Peugeot* (No et ville d'agence)

R5 : Que produit la requête suivante ?

```
SELECT Voit_Puissance, Voit_DateAchat, Ag_No, Ag_Ville
FROM Voiture, Agence
WHERE Voit_DateAchat<#1/1/2000#
AND Ag_No=Voit_NoAg#
```

Correction

R1 : SELECT Voit_NoImmat

FROM Voiture, Marque

WHERE Mrq_Nom="Renault"

AND Mrq_No=Voit_NoMrq#

R2 : SELECT Voit_NoImmat, Voit_Puissance

FROM Voiture

WHERE Voit_Puissance>6

AND Voit_NoAg#=4

R3 : SELECT Ag_No, Ag_Ville

FROM Agence, Voiture, Marque

WHERE Mrq_Nom="Peugeot"

AND Mrq_No=Voit_NoMrq#

AND Voi_NoAg#=Ag_No

R4 : Impossible pour l'instant, car la jointure ne conserve que les agences possédant un véhicule

R5 : Liste des puissances, dates d'achat, n° et villes des agences ayant des voitures achetées avant l'an 2000

5. Trier les données affichées

Clause de tri : ORDER BY *Colonne1* [DESC], *Colonne2* [DESC], ...

Une requête sans clause de tri affiche les données comme bon lui semble. Un tri sera composé de noms de colonnes et éventuellement de clause DESC pour inverser l'ordre naturel.

Colonne1 est le critère le plus important (critère majeur), *colonne2* est facultative et constitue un critère mineur. Si *colonne1* est identique, l'ordre sera donné par *colonne2*. Etc.

Exemple :

Liste des sections par ordre alphabétique



```
SELECT Sect_No, Sect_Nom
FROM Section
ORDER BY Sect_Nom
```

Exercice

Ecrire les requêtes suivantes

R1 : Liste des étudiants par section puis par nom
 R2 : Liste des étudiants par nom et prénom
 R3 : Liste des étudiants par date de naissance (le plus jeune en premier)
 R4 : Liste des étudiants par date de naissance (le plus âgé en premier)

Correction

```
R1 :  SELECT Etu_No, Etu_Nom, Sect_Nom
      FROM Etudiant, Section
      WHERE Sect_No=Etu_NoSect#
      ORDER BY Sect_Nom, Etu_Nom
R2:  SELECT Etu_No, Etu_Nom, Etu_Prénom
      FROM Etudiant
      ORDER BY Etu_Nom, Etu_Prénom
R3:  SELECT Etu_Nom, Etu_Prénom, Etu_DateNais
      FROM Etudiant
      ORDER BY Etu_DateNais DESC
R4:  SELECT Etu_Nom, Etu_Prénom, Etu_DateNais
      FROM Etudiant
      ORDER BY Etu_DateNais
```

A noter qu'un ordre de tri est lié à ce qui est affiché, donc les colonnes du tri doivent être dans le SELECT

6. Fonctions de regroupement

Elles permettent de faire des calculs faisant intervenir les différentes lignes affichées. Jusqu'ici, les opérations entre colonnes se limitaient aux opérations arithmétiques simples (+-* /).

Exemple Produit (Réf, Nom, PV, PA)

Pour obtenir la marge : SELECT Réf, Nom, PV-PA as Marge
 FROM Produit

Fonction de décompte : COUNT (*)

Elle renvoie le nombre de lignes qui étaient affichées en mode détail.

Exemple :

Nombre de sections



```
SELECT COUNT (*)
FROM Section
```

2ème exemple : Combien d'étudiants sont en SIO1 ?

```
SELECT COUNT(*)
FROM Etudiant, Section
WHERE Sect_Nom="SIO1"
AND Sect_No=Etu_NoSect#
```


Noter :

- 1.) Que l'on ne mélange pas une fonction de regroupement avec une colonne détail dans le SELECT ;
- 2.) Qu'en cas de présence d'une clause WHERE, la fonction de regroupement est impactée.

Fonction de maximum : MAX (Colonne)

Elle renvoie la valeur « maximale » de la colonne parmi les lignes qui étaient affichées en mode détail. « Maximale » s'applique naturellement aux colonnes numériques, mais aussi texte (ZZ...), date (plus récente) ou booléennes (?).

Exemple :

Date de naissance de l'étudiant le moins âgé (date de naissance « maximale »)



```
SELECT MAX (Etu_DateNais)
FROM Etudiant
```

Date de naissance de l'étudiant le moins âgé parmi les SIO1

```
SELECT MAX (Etu_DateNais)
FROM Etudiant, Section
WHERE Sect_Nom="SIO1"
AND Sect_No=Etu_NoSect#
```

Fonction de minimum : MIN (Colonne)

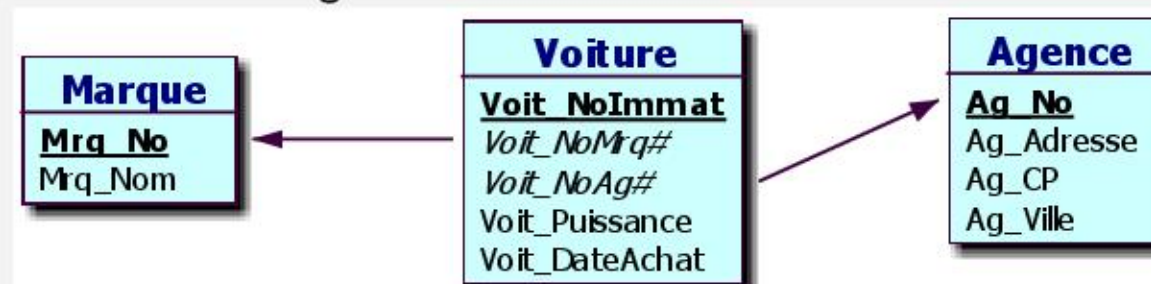
Pareil dans l'autre sens.

Fonction de moyenne : AVG (Colonne)

Elle renvoie la valeur moyenne de la colonne parmi les lignes qui étaient affichées en mode détail. Moyenne n'a de sens que pour des données numériques.

Exemple :

Puissance moyenne des véhicules de l'agence n° 8



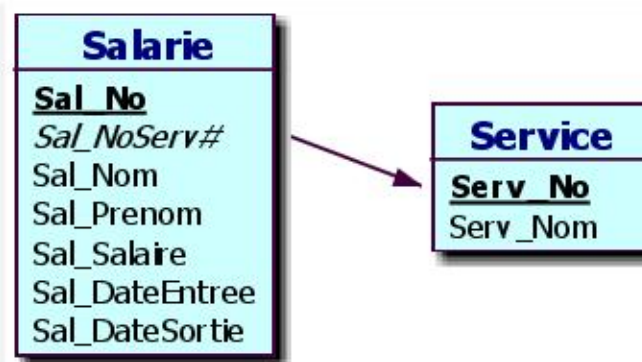
```
SELECT AVG (Voit_Puissance)
FROM Voiture
WHERE Voit_NoAg#=8
```

Fonction de cumul : SUM (Colonne)

Elle renvoie le total de la colonne parmi les lignes qui étaient affichées en mode détail. Sum ne s'applique qu'aux colonnes numériques.

Exemple :

Total des salaires des employés



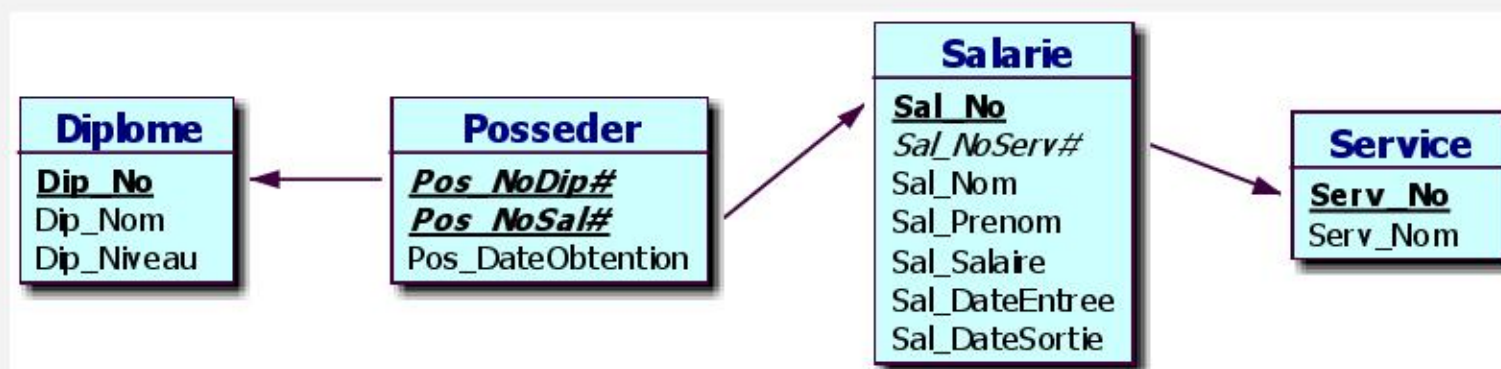
```

SELECT SUM (Sal_Salaire)
FROM Salarié

```

Exercice

Affectation et diplômes des salariés



Produire les requêtes suivantes :

R1 : Combien de salariés ont un diplôme de niveau au moins égal à 2

R2 : Quel est le niveau de diplôme le plus élevé

R3 : Parmi les diplômes détenus, quel est le niveau de diplôme le plus élevé

R4 : Quel est le niveau de diplôme moyen des salariés du service *Compta*

Correction

```

R1 :  SELECT COUNT (*)
      FROM Salarié, Posséder, Diplôme
      WHERE Dip_Niveau >= 2
            AND Dip_No = Pos_NoDip#
            AND Pos_NoSal# = Sal_No
R2 :  SELECT MAX (Dip_Niveau)
      FROM Diplôme
R3 :  SELECT MAX (Dip_Niveau)
      FROM Diplôme, Posséder, Salarié
      WHERE Dip_No = Pos_NoDip#
            AND Pos_NoSal# = Sal_No
R4 :  SELECT AVG (Dip_Niveau)
      FROM Diplôme, Posséder, Salarié, Service
      WHERE Serv_Nom = "Compta"
            AND Dip_No = Pos_NoDip#
            AND Pos_NoSal# = Sal_No
            AND Sal_NoServ# = Serv_No

```

7. Regroupement

Les fonctions de regroupement vues avant s'appliquent en une seule fois jusqu'à présent. On peut les faire appliquer par parties (sommations par parties (sous-totaux), décomptes, minima, maxima, ...).

Quand le résultat fait appel à des fonctions de regroupement, mais qu'il peut posséder plusieurs lignes, c'est qu'il faut un regroupement.

Regroupement : GROUP BY *Colonne1, Colonne2, ...*

Colonne1, Colonne2 font partie de la clause SELECT. Leur ordre n'a pas d'incidence.

Noter :

- 1.) Il ne faut surtout pas confondre cette clause avec celle du tri, qui est sans rapport !
- 2.) Cette clause s'utilise toujours avec au moins une fonction de regroupement (Count, Sum, Min, Max, Avg).

Exemple :

Nombre d'étudiants par section

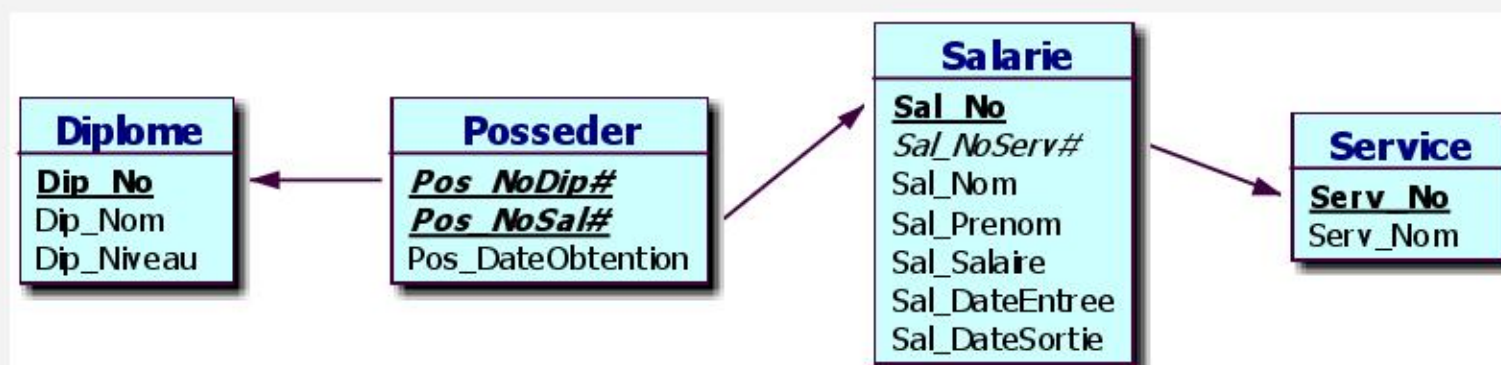


```
SELECT COUNT (*), Sect_No, Sect_Nom
FROM Etudiant, Section
WHERE Etu_NoSect#=Sect_No
GROUP BY Sect_No, Sect_Nom
```

Règle : Quand la requête mélange détail (ici Sect_Nom) et fonction de regroupement (ici Count), un *Group By* est obligatoire et doit porter sur toutes les colonnes du détail.

Exercice

Affectation et diplômes des salariés



Produire les requêtes suivantes :

- R1 : Niveau de diplôme maximal des salariés dans chacun des services
- R2 : Quel est le salaire minimal et maximal par service
- R3 : Quel est le nombre et la date du dernier diplôme de chaque salarié
- R4 : Quel est l'effectif de chaque service
- R5 : Que produit la requête suivante ?

```
SELECT Dip_Nom, MAX (PosDateObtention)
FROM Diplôme, Posséder
WHERE Dip_No=Pos_NoDip#
GROUP BY Dip_Nom
```

Correction

```
R1 : SELECT Serv_No, Serv_Nom, MAX (Dip_Niveau)
      FROM Salarié, Posséder, Diplôme, Service
      WHERE Dip_No=Pos_NoDip#
            AND Pos_NoSal#=Sal_No
            AND Sal_NoServ#=Serv_No
      GROUP BY Serv_No, Serv_Nom
R2 : SELECT Serv_No, Serv_Nom, MAX (Sal_Salaire), MIN (Sal_Salaire)
      FROM Salarié, Service
      WHERE Sal_NoServ#=Serv_No
      GROUP BY Serv_No, Serv_Nom
R3 : SELECT Sal_No, Sal_Nom, Sal_Prénom, COUNT (*), MAX (Pos_DateObtention)
      FROM Posséder, Salarié
      WHERE Pos_NoSal#=Sal_No
      GROUP BY Sal_No, Sal_Nom, Sal_Prénom
R4 : SELECT Serv_No, Serv_Nom, COUNT(*)
      FROM Salarié, Service
      WHERE Sal_NoServ#=Serv_No
      GROUP BY Serv_No, Serv_Nom
R5 : Les dates de dernière obtention dans chaque diplôme. On ne sait pas qui les a eus !
```

8. Regroupement avec condition

On peut faire porter un critère sur chaque regroupement opéré précédemment, pour n'en retenir que certains, comme si un *Where* était placé avec le *Group by*.

Regroupement avec condition :

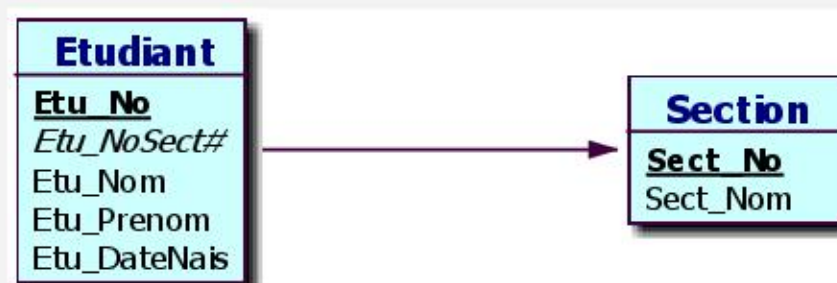
```
GROUP BY Colonne1, Colonne2, ...
HAVING Critère(s)
```

Noter :

- 1.) La clause *Having* ne peut que supprimer des lignes du résultat
- 2.) Cette clause porte sur les lignes regroupées par le *Group by*. On y trouve toujours une fonction de regroupement (Count, Sum, Min, Max, Avg).

Exemple :

Sections comportant au moins 20 étudiants



```
SELECT Sect_No, Sect_Nom, COUNT (*) as Nb
FROM Etudiant, Section
WHERE Etu_NoSect#=Sect_No
GROUP BY Sect_No, Sect_Nom
HAVING COUNT (*)>=20
```

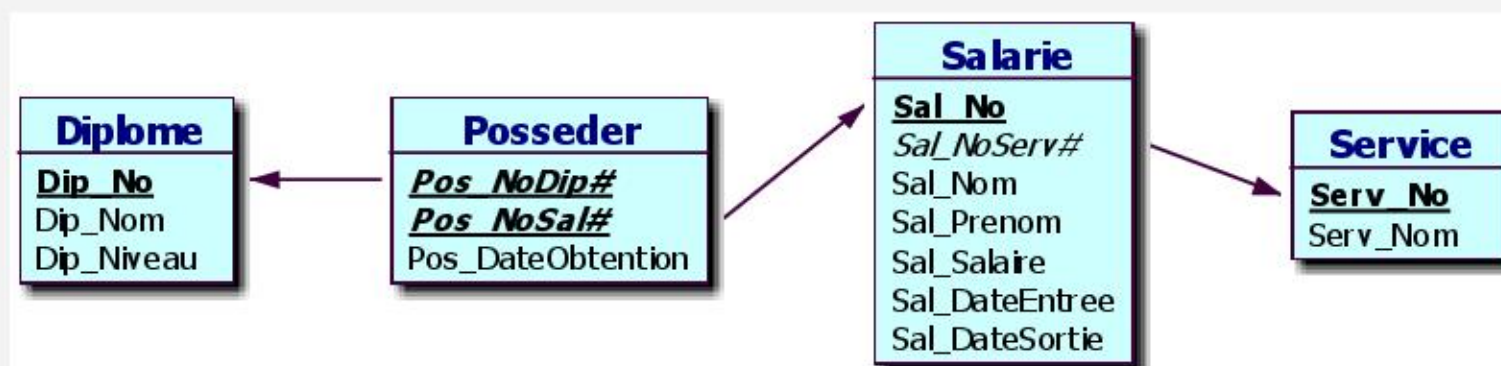

Remarque : Les deux fonctions *Count(*)* portent sur les mêmes lignes. Elles rendent le même résultat. On aurait pu écrire :

```
SELECT Sect_No, Sect_Nom, COUNT (*) as Effectif
FROM Etudiant, Section
WHERE Etu_NoSect#=Sect_No
GROUP BY Sect_No, Sect_Nom
HAVING Effectif>=20
```

Astuce à retenir : Dès qu'une condition porte sur une fonction de regroupement, la condition n'est plus dans le *Where* mais dans la clause *Having* du *Group By*.

Exercice

Affectation et diplômes des salariés



Produire les requêtes suivantes :

- R1 : Services ayant un salaire moyen au delà de 2 000 €
- R2 : Salariés possédant au moins 2 diplômes
- R3 : Salariés possédant au moins 2 diplômes de niveau 3 ou plus

Correction

- R1 :

```
SELECT Serv_No, Serv_Nom, AVG (Sal_Salaire) As SalaireMoyen
FROM Salarié, Service
WHERE Sal_NoServ#=Serv_No
GROUP BY Serv_No, Serv_Nom
HAVING SalaireMoyen>2000
```
- R2 :

```
SELECT Sal_No, Sal_Nom, COUNT (*) as NbDiplômes
FROM Salarié, Posséder, Diplôme
WHERE Pos_NoSal#=Sal_No
AND Pos_NoDip#=Dip_No
GROUP BY Sal_No, Sal_Nom
HAVING NbDiplômes>=2
```
- R3 :

```
SELECT Sal_No, Sal_Nom, COUNT (*) as NbDiplômes
FROM Salarié, Posséder, Diplôme
WHERE Dip_Niveau>=3
AND Dip_No=Pos_NoDip#
AND Pos_NoSal#=Sal_No
GROUP BY Sal_No, Sal_Nom
HAVING NbDiplômes>=2
```

9. Conditions d'appartenance / non appartenance à un ensemble



Pour trouver les sections sans étudiant, l'idée est de procéder en 2 étapes :

- 1. On cherche le(s) n° de section **avec** étudiants. On obtient une liste de valeurs.
- 2. On cherche les sections dont le n° de section ne fait pas partie de cette liste.

Dès qu'on a besoin de cette logique de **liste de valeurs** à accepter (respectivement refuser), il faudra utiliser l'opérateur *IN* (respectivement *NOT IN*).

Première requête à exécuter :	Deuxième requête à exécuter :
<pre>SELECT Etu_NoSect# FROM Etudiant</pre> Rendra 2, 4, 6 par exemple	<pre>SELECT * FROM Section WHERE Sect_No NOT IN (<Liste de no de section>)</pre> Par exemple 2, 4, 6

La solution :

Placer la requête finale (le résultat à produire devant), et se servir de l'autre ensuite :

Pour trouver les sections sans personne :

```
SELECT *
FROM Section
WHERE Sect_No NOT IN ( SELECT Etu_NoSect#
                        FROM Etudiant )
```

Autre façon d'utiliser le *IN* : Pour énumérer une liste de valeur (évite des comparaisons « = » avec des *OR*).
Exemple Liste des étudiants prénommés Luc, René ou Léa :

Avec le IN	Sans le IN
<pre>SELECT Etu_No, Etu_Nom, Etu_Prénom FROM Etudiant WHERE Etu_Prénom IN ("Luc", "René", "Léa")</pre>	<pre>SELECT Etu_No, Etu_Nom, Etu_Prénom FROM Etudiant WHERE Etu_Prénom = "Luc" OR Etu_Prénom = "René" OR Etu_Prénom = "Léa"</pre>

Exercice :



Afficher les sections sans étudiant.

Correction :

```
SELECT Sect_No, Sect_Nom
FROM Section
WHERE Sect_No NOT IN (SELECT Etu_NoSect#
                      FROM Etudiant)
```


10. Clause **DISTINCT**

Rarement utilisée, elle permet dans le *SELECT* que chaque ligne affichée soit « unique », sans doublon. Cette clause dépend donc des colonnes projetées (affichées), et consiste à enlever les répétitions. Exemple : Liste des prénoms des étudiants :

```
SELECT Etu_Prénom  
FROM Etudiant
```

Robert pourra apparaître plusieurs fois.

```
SELECT DISTINCT Etu_Prénom  
FROM Etudiant
```

Chaque prénom sera unique

DISTINCT accepte un synonyme : *DISTINCT ROW*.

A savoir : SQL accepte encore d'autres primitives, mais tous les SGBD ne les supportent pas nécessairement. Pour en contourner certaines, dans Access par exemple, il est parfois nécessaire d'écrire deux requêtes séparées.